

OpenCollab 各模式的简单介绍

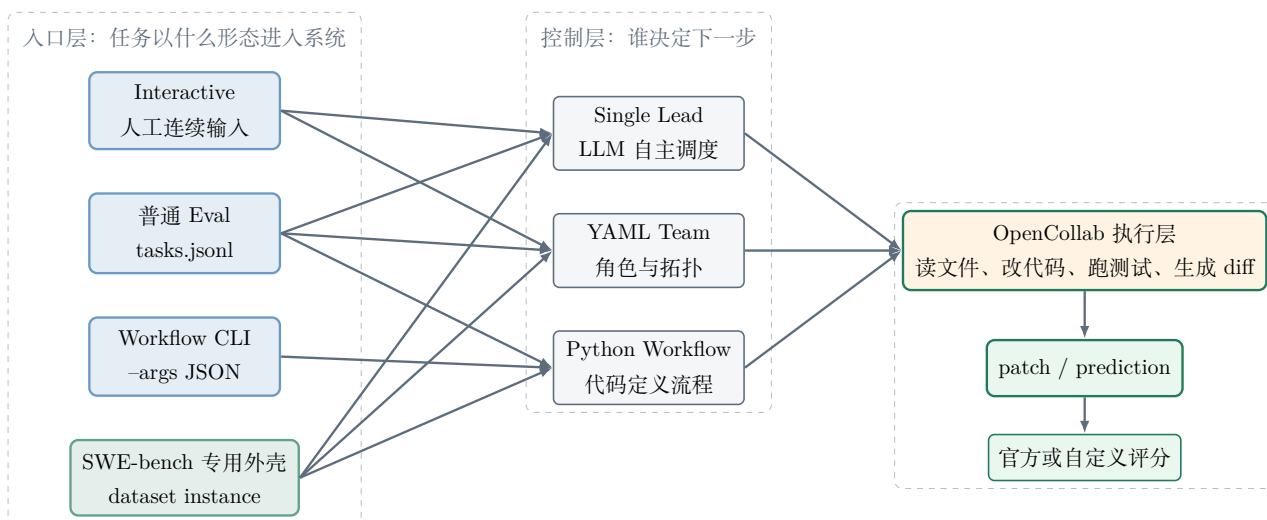
2026-06-29

1 概述

OpenCollab 的使用方式可以分成两层来看。第一层是任务怎么进入系统，包括交互式输入、普通 Eval、Workflow CLI，以及为 SWE-bench 写的专用 Eval 外壳。第二层是进入系统之后由谁决定下一步，包括单 Lead、YAML Team 和 Python Workflow。

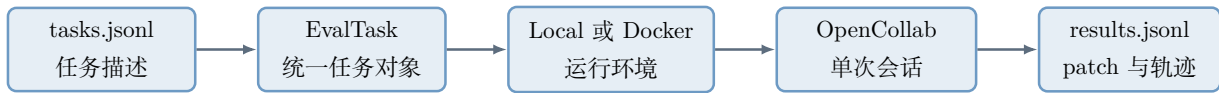
普通 Eval 是一个很薄的批量任务外壳。它读取 JSONL，把每一行变成一个 EvalTask，给 OpenCollab 一个任务描述和一个可选运行环境，最后提取 git diff 写进结果文件。SWE-bench 专用 Eval 外壳更厚。它额外处理 SWE-bench 数据集、官方镜像、/testbed 容器、问题描述、隐藏测试字段、预测文件和官方评分器。两者都可以驱动 OpenCollab，但面向的任务协议不同。

2 总图



3 普通 Eval 外壳

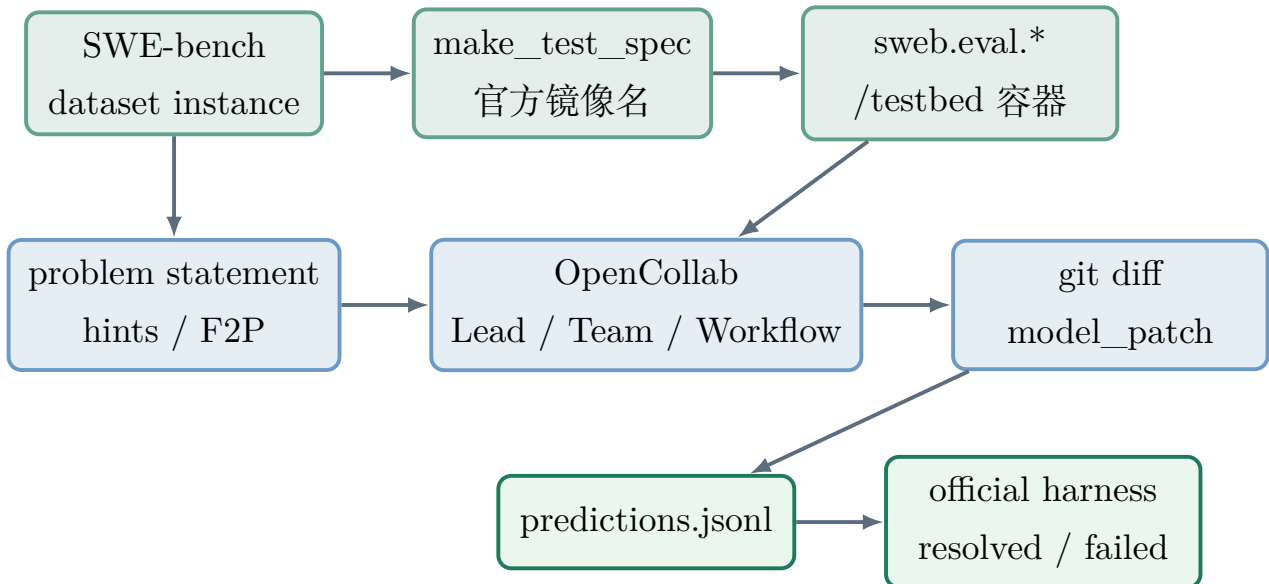
普通 Eval 的命令形态是 `opencollab eval tasks.jsonl`。这个 JSONL 文件每行代表一个任务，核心字段是 `task_id` 和 `description`，可选字段包括 `repo_path`、`docker_image`、`timeout` 和 `max_tokens`。CLI 读入这些字段之后构造 EvalTask，再交给通用 evaluator 执行。



普通 Eval 的职责边界很清楚。它负责批量读取任务、准备本地或 Docker 环境、发送一次初始任务描述、运行 OpenCollab、提取最终 diff、保存通用结果。它不理解 SWE-bench 的官方镜像命名，不解析 `problem_statement`，不生成 SWE-bench 官方 `predictions.jsonl`，也不直接调用官方评分器。

4 SWE-bench 专用 Eval 外壳

SWE-bench 专用外壳的输入不是普通 JSONL 任务，而是 SWE-bench 数据集实例。每个实例包含 `instance_id`、`repo`、`problem_statement`、`hints_text`、`FAIL_TO_PASS` 等 benchmark 字段。外壳先用 SWE-bench 官方工具确定实例镜像，再在官方 `/testbed` 环境中启动 OpenCollab。



这套外壳做的是 benchmark 适配。它把 SWE-bench 自己的任务格式、镜像格式、评测格式转换成 OpenCollab 的一次解题过程，再把 OpenCollab 产生的 diff 转回 SWE-bench 官方预测格式。我们之前主要使用的 Team 评测路径，就是由 `scripts/run_team_batch.sh` 选择实例并解析镜像，再由 `scripts/start_team_run.sh` 启动容器、构造 prompt、调用 OpenCollab。